

Winson Lin, Evan Ho, Katherine Han, Hajime Kioi

Dr. Kemere

ELEC 327

6 May 2026

Project Report

Concept

This project was inspired by the “OIIA OIIA” spinning cat meme, a viral TikTok trend featuring a green-screened cat spinning to a catchy electronic song. Our goal was to recreate the meme physically by synchronizing a spinning motor and LED animations with the music.

To play the audio, we use an ESP32 microcontroller to read the song from a microSD card and output the signal through an amplifier and speaker. The MSPM0 microcontroller controls the motor and LED ring, allowing the visual effects and spinning motion to match the timing of the music.

Architecture

The implemented architecture consists of three main subsystems: power delivery, off-board computation, and the on-board system. The power delivery subsystem provides the required supply voltages for the microcontrollers and devices. The off-board computation subsystem is responsible for preprocessing the audio, motor, and LED timing data before deployment. The on-board system contains the ESP32 and MSPM0G3507 microcontrollers, which coordinate audio playback, motor control, and LED animation during operation.

A. Power Delivery

The power delivery subsystem uses USB-C as the main input and regulates it into the voltage rails required by the system. It negotiates 12V at 5A for the motor and the downstream bucks. A 5 V rail powers the ESP32, audio amplifier, and an LED ring, while a 3.3 V rail powers lower-voltage logic such as the MSPM0G3507 and the microSD reader. This separation helps isolate sensitive control electronics from larger current spikes caused by the motor, LEDs, and speaker. Decoupling capacitors and short power routing were used to improve voltage stability during playback and motor actuation.

B. Off-board computation

Motor and LED control data were generated from audio using Python and the Librosa audio analysis library, then exported as .h files for the MSPM0 microcontroller. For the motor system, RMS energy, beat detection, and onset detection were used to generate synchronized PWM values. For the LED system, RMS energy controlled comet brightness, speed, and tail length, while onset detection triggered color changes. The processed animation and PWM data were stored as compact frame arrays in header files for efficient real-time playback on the embedded system.

C. On-board system

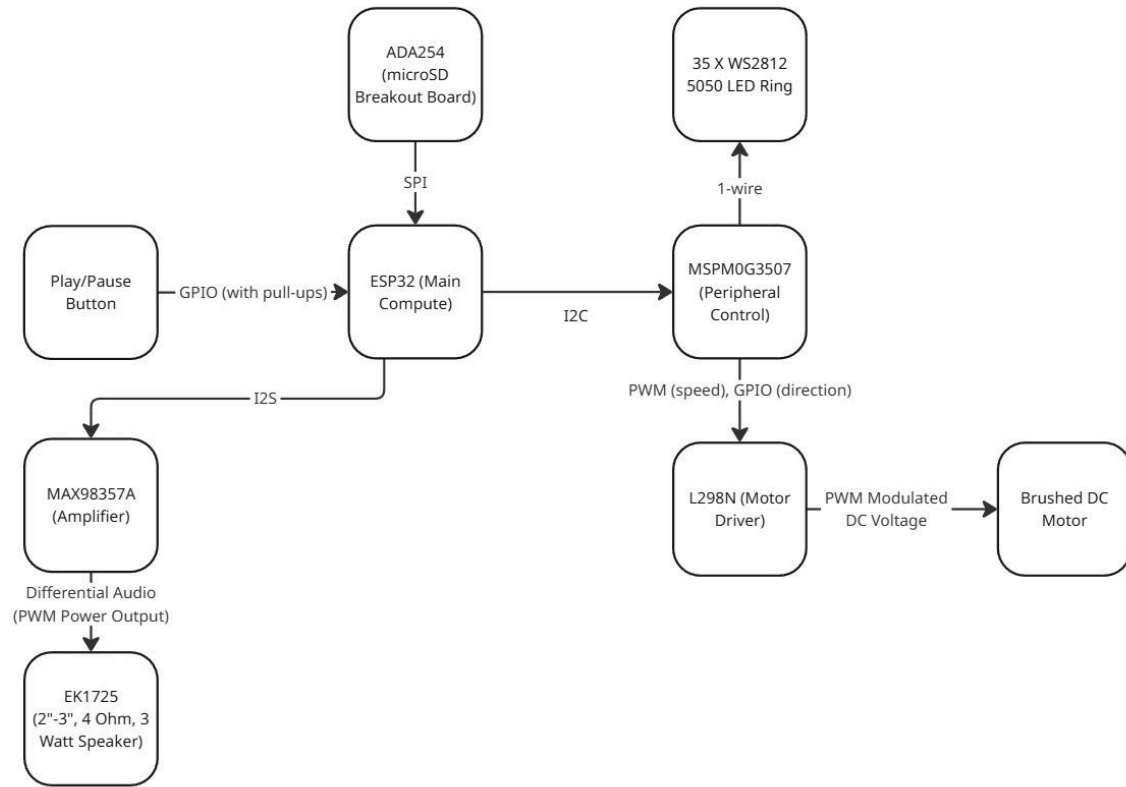


Figure 1. On-board System Architecture

The on-board system architecture consists of two microcontrollers: an ESP32, which acts as the main compute and audio playback controller, and an MSPM0G3507, which handles peripheral control for the motor and LED ring. Figure 1 shows the overall system architecture and communication between each subsystem.

The ESP32 is responsible for reading audio data (.wav) from the microSD breakout board through SPI. A play/pause button is connected to the ESP32 through GPIO, allowing the user to control playback. During operation, the ESP32 outputs the audio stream to the MAX98357A amplifier using the I2S protocol. The amplifier then drives the EK1725 speaker using a differential audio PWM output.

The ESP32 also communicates with the MSPM0G3507 using I2C. The MSPM0G3507 acts as the peripheral controller for the visual and motion effects. It drives the 35-LED WS2812 5050 LED ring using a one-wire data interface. For motion control, the MSPM0G3507 sends a PWM signal and direction GPIO signal to the L298N motor driver. The L298N then outputs a PWM-modulated DC voltage to the brushed DC motor, allowing the spinning motion to be synchronized with the LED effects and music timing.

Sensors & Devices Used

MSPM0G3507

The MSPM0G3507 was the real-time control microcontroller for the project. It receives play/pause commands over I2C from the ESP32 and drives the LED ring and DC motor.

ESP32

The ESP32 is used as the media and communication controller. It reads the audio file from the MicroSD card, outputs digital audio to the amplifier using I2S, monitors the play/pause button input, and sends simple control commands to the MSPM0 over I2C.

MAX98357A I2S Audio Amplifier

The MAX98357A converts the ESP32's I2S audio stream into an amplified signal for the speaker.

EK1725 4Ω 3W Speaker

The EK1725 speaker is the audio output device for the project, connected to the amplifier.

ADA254 MicroSD Card Breakout

The ADA254 MicroSD Card Breakout provides storage for the project's audio WAV file. It interfaces with the ESP32 over SPI.

L298N Motor Driver

The L298N motor driver is used to control the brushed DC motor from the MSPM0. It received direction and PWM control signals from the MSPM0 through GPIO pins.

Brushed DC Motor (5V 6000RPM)

The 5V brushed DC motor provides the rotational motion for the spinning cat mechanism. Its speed is controlled by the MSPM0 through the motor driver using PWM-based speed control.

35× WS2812 LED Ring:

The 35-LED WS2812 ring is controlled by the MSPM0 using a one-wire digital interface. The MSPM0 sends a timed serial data signal over a single GPIO pin to control the entire ring.

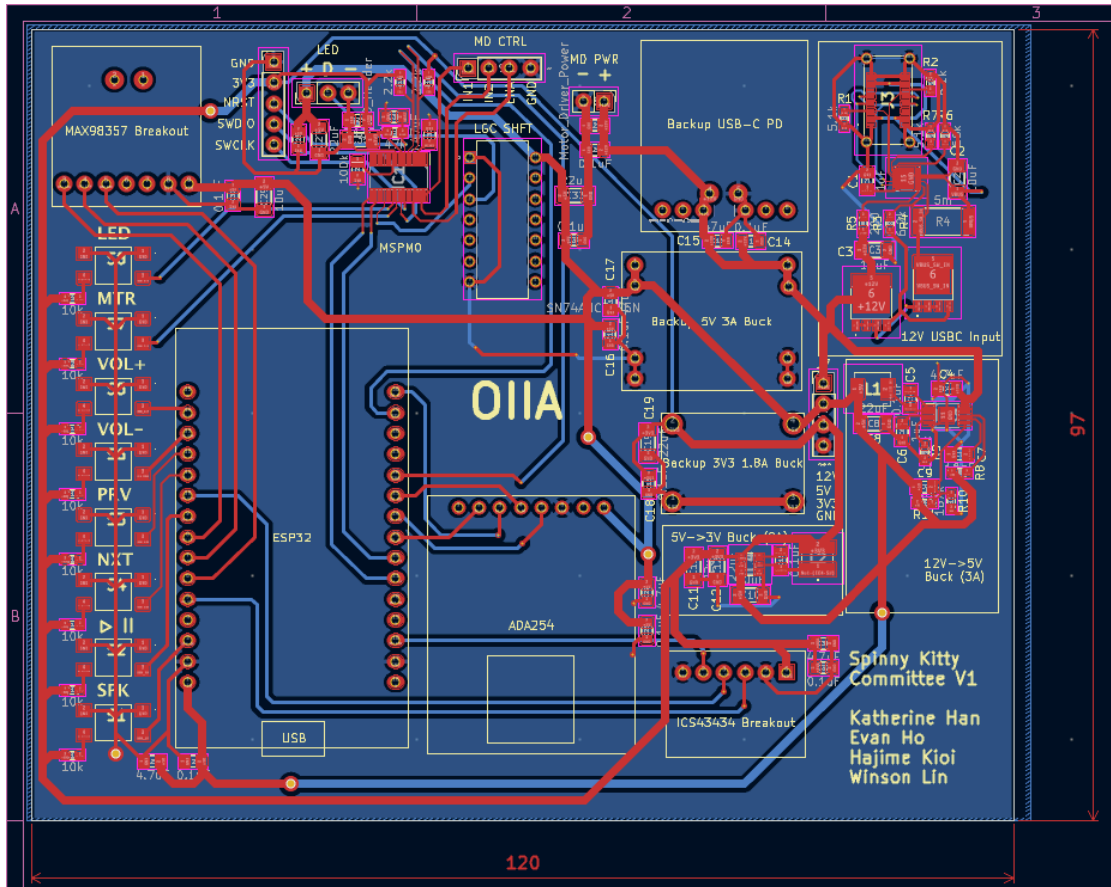
Complexity Bonuses

A. SLOC Count

SLOC count: 747

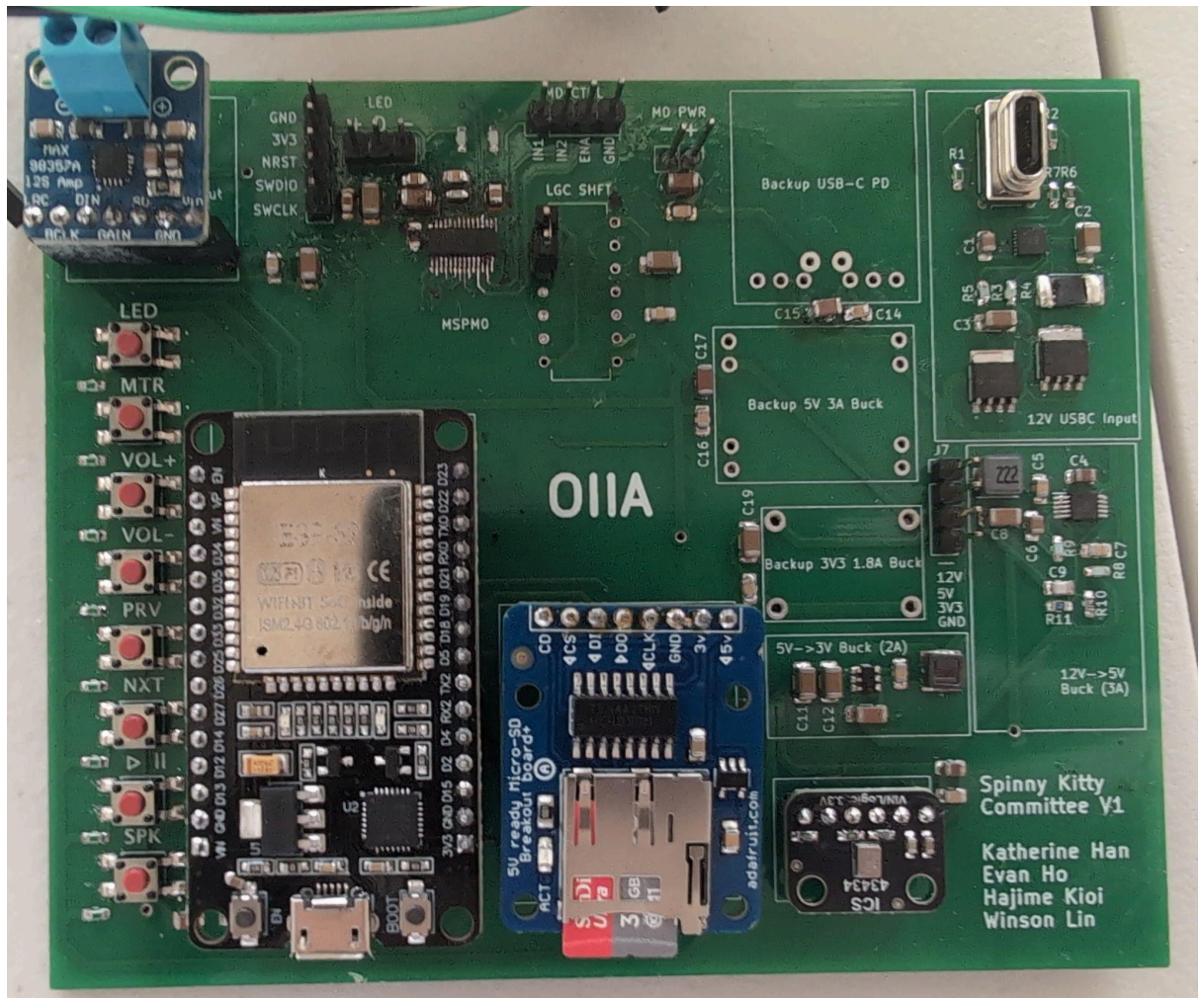
Language	files	blank	comment	code
Python	1	122	104	392
C	1	85	70	314
Markdown	1	14	0	41
SUM:	3	221	174	747

B. PCB design complexity



We have implemented a simple board with power, digital I/O, and analog signals (ESP32, audio amplifier).

C. Integration



D. Github Repo and Website

Github Repo: <https://github.com/winsonlin8/SpinnyKittyCommittee.git>

Project Website: spinnykitty.github.io